

Table of Contents

1. Introduction	3
1.1 What is TeamForge IDE?	3
1.2 Key Features	3
1.3 System Requirements	3
2. Getting Started	4
2.1 Installation	4
2.2 First Launch	4
2.3 Interface Overview	4
3. AI Agents	5
3.1 Overview of the Six Agents	5
3.2 Agent Statuses	5
3.3 Agent Detail Dialog	6
3.4 YOLO Mode	6
3.5 Agent Orchestrator and Scheduler	6
4. AI Chat System	7
4.1 Context-Aware AI	7
4.2 Multi-Provider Support	7
4.3 Chat Sessions	7
4.4 Slash Commands	8
5. Code Editor	8
5.1 Multi-File Tabs	8
5.2 Syntax Highlighting	8
5.3 Find and Replace	9
5.4 Navigation	9
5.5 Run Current File	9
5.6 Auto-Save	9

6. Project Management	10
6.1 Virtual File System	10
6.2 Task Board	10
6.3 Task Priority Levels	10
6.4 Task Types	11
6.5 Auto-Assignment	11
7. Terminal and Build	11
7.1 Interactive Terminal	11
7.2 Build, Lint, and Test Commands	11
7.3 Pre-Deploy Checks	12
7.4 Command Whitelist	12
8. Settings and Configuration	12
8.1 AI Provider Settings	12
8.2 Editor Preferences	12
8.3 API Key Configuration	13
8.4 Connection Testing	13
9. Keyboard Shortcuts	13
10. API Reference	14
11. AI Providers Reference	16
11.1 Z-AI	16
11.2 NVIDIA NIM	16
11.3 OpenAI-Compatible Endpoints	16
12. Troubleshooting	17
12.1 Common Issues and Solutions	17
12.2 WebSocket Connection Issues	17
12.3 AI Provider Fallback	18

1. Introduction

1.1 What is TeamForge IDE?

TeamForge IDE is a next-generation autonomous agentic integrated development environment designed to revolutionize software development workflows. Built with a modern web-based architecture using Next.js, React, and Tailwind CSS, it combines the power of multiple specialized AI agents with a full-featured code editor, context-aware chat system, and comprehensive project management tools. Unlike traditional IDEs that rely solely on human input, TeamForge IDE leverages a team of six AI agents that collaborate autonomously to plan, architect, develop, review, test, and deploy code.

1.2 Key Features

Multi-Agent AI System -- Six specialized agents (Nova, Atlas, Codey, Prism, Flux, Blaze) work collaboratively on tasks, each with distinct roles covering the full software development lifecycle.

Context-Aware AI Chat -- The chat system understands project structure, active files, dependencies, and build output to provide highly relevant assistance. Supports multiple AI providers including Z-AI, NVIDIA NIM, and OpenAI-compatible endpoints.

Intelligent Code Editor -- A multi-file editor with syntax highlighting, find and replace, go-to-line navigation, and the ability to run files directly with automatic language detection.

Project Management -- Integrated task board with six status columns, priority levels, task types, and automatic agent assignment based on task type and agent specialization.

Interactive Terminal -- Execute commands directly within the IDE with build, lint, and test integration, pre-deploy checks, and a command whitelist for safety.

Virtual File System -- Full file and directory management within the browser, supporting create, edit, delete, and batch operations without requiring local file system access.

1.3 System Requirements

Requirement	Specification
Node.js	18.x or later
Bun Runtime	1.x or later (recommended)
Database	PostgreSQL (via Prisma ORM)
Operating System	Linux, macOS, or Windows with WSL2
Memory	Minimum 4 GB RAM (8 GB recommended)
Disk Space	500 MB for application + dependencies
Browser	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

Table 1: System Requirements

2. Getting Started

2.1 Installation

TeamForge IDE can be set up locally by following these steps:

1. **Clone the repository:** git clone the project repository to your local machine.
2. **Install dependencies:** Run `bun install` to install all required packages.
3. **Initialize the database:** Run `bun run db:push` to set up the PostgreSQL schema via Prisma.
4. **Start the development server:** Run `bun run dev` to launch the application.

```
$ git clone <repository-url>
$ cd teamforge-ide
$ bun install
$ bun run db:push
$ bun run dev
```

2.2 First Launch

Upon first launch, the application opens in your default web browser at `http://localhost:3000`. The initial view presents the dashboard with the sidebar navigation on the left, the main workspace area in the center, and the AI chat panel on the right. You will see the six AI agents displayed in the sidebar, each in an idle state, ready to begin work.

2.3 Interface Overview

The TeamForge IDE interface is organized into a three-column layout that maximizes productivity by keeping all essential tools visible simultaneously:

Left Sidebar: Contains the file explorer (virtual file system), AI agents panel, task board, and navigation between different workspace views including dashboard, agents, tasks, chat, terminal, and settings.

Center Editor: The main workspace area features a tabbed code editor with syntax highlighting, breadcrumb navigation, find and replace functionality, and a bottom panel for terminal output, build logs, and system logs.

Right Chat Panel: The AI chat interface provides context-aware assistance with support for slash commands, multi-session management, and real-time streaming responses from your selected AI provider.

3. AI Agents

3.1 Overview of the Six Agents

TeamForge IDE employs six specialized AI agents, each designed to handle a specific aspect of the software development lifecycle. These agents collaborate through an orchestration system that assigns tasks, manages dependencies, and ensures smooth handoffs between phases.

Agent	Role	Responsibilities
Nova	Project Manager	Task planning, prioritization, resource allocation, and project coordination. Breaks down requirements into actionable tasks.
Atlas	Architect	System design, technical architecture decisions, technology selection, and design pattern recommendations.
Codey	Developer	Code implementation, feature development, bug fixes, and technical debt resolution. The primary code-writing agent.
Prism	Reviewer	Code review, quality assurance, best practices enforcement, and security vulnerability detection.
Flux	Tester	Test creation, test execution, coverage analysis, and regression detection. Ensures code reliability.
Blaze	DevOps	Deployment automation, infrastructure management, CI/CD pipeline configuration, and monitoring setup.

Table 2: AI Agent Overview

3.2 Agent Statuses

Each agent can be in one of several states, providing real-time visibility into their current activity and availability:

Status	Description	Visual Indicator
Idle	Agent is available and waiting for task assignment	Gray dot
Thinking	Agent is processing a task or analyzing code	Pulsing yellow dot
Coding	Agent is actively writing or modifying code	Green dot
Reviewing	Agent is reviewing code for quality and correctness	Blue dot
Testing	Agent is creating or executing tests	Orange dot
Deploying	Agent is managing deployment operations	Purple dot

Sleeping	Agent is inactive due to inactivity timeout	Dimmed gray dot
----------	---	-----------------

Table 3: Agent Status States

3.3 Agent Detail Dialog

Clicking on any agent opens the Agent Detail Dialog, which provides comprehensive information about the selected agent including: current status and active task, task history and completion statistics, specialized capabilities and preferred task types, recent activity log with timestamps, and performance metrics such as average task completion time and success rate.

3.4 YOLO Mode

YOLO Mode is TeamForge IDE's full autonomy mode. When activated, the AI agents operate with minimal human oversight, automatically proceeding through the entire development lifecycle from planning to deployment. In YOLO Mode: Nova plans and assigns tasks automatically, Atlas makes architectural decisions independently, Codey implements features without requiring approval for each change, Prism performs code reviews and can approve changes autonomously, Flux runs tests and reports results, and Blaze handles deployment when all quality gates pass.

Warning: YOLO Mode is intended for experienced users who trust the AI system. It is recommended to use standard mode for critical production code and enable YOLO Mode only for prototyping, experimentation, or well-tested workflows.

3.5 Agent Orchestrator and Scheduler

The Agent Orchestrator is the central coordination system that manages task assignment and inter-agent communication. It operates through the following mechanisms:

Task Assignment: Automatically assigns tasks to the most appropriate agent based on task type, agent specialization, and current workload.

Dependency Management: Tracks task dependencies and ensures that prerequisite tasks are completed before dependent tasks begin.

Handoff Protocol: Manages smooth transitions between agents, passing context and artifacts (code, test results, review comments) between phases.

Conflict Resolution: Handles situations where multiple agents need to modify the same file simultaneously through a locking and merging mechanism.

Scheduler: The agent scheduler API endpoint allows external systems to trigger agent actions, schedule periodic tasks, and monitor agent health.

4. AI Chat System

4.1 Context-Aware AI

The AI chat system in TeamForge IDE is deeply integrated with your project context. Unlike generic chat interfaces, it automatically incorporates the following context into every conversation:

Project Structure: The chat engine understands your entire project layout, including file hierarchy, directory conventions, and module organization.

Active File: The currently open file in the editor is automatically included in the chat context, allowing the AI to reference and modify it directly.

Dependencies: Package dependencies from package.json and other configuration files are considered when providing recommendations or generating code.

Build Output: Recent build errors, warnings, and test results are available to the AI for debugging and fix suggestions.

4.2 Multi-Provider Support

TeamForge IDE supports three categories of AI providers, offering flexibility in choosing the model that best fits your needs and budget:

Provider	Models	Key Features
Z-AI	GLM-4, GLM-4 Flash	Optimized for code generation and analysis. Low latency with Flash variant.
NVIDIA NIM	80+ models	Access to Meta Llama, NVIDIA Nemotron, DeepSeek, Mistral, Qwen, Gemma, Phi, and more.
OpenAI-Compatible	Varies by endpoint	Connect to Ollama, LM Studio, vLLM, Together AI, or any OpenAI-compatible API.

Table 4: AI Provider Overview

4.3 Chat Sessions

The chat system supports multiple concurrent sessions, each with its own context and history:

Create Session: Start a new chat session from the chat panel header. Each session maintains independent conversation history.

Rename Session: Double-click the session name to rename it for easy identification.

Switch Session: Click on any session in the session list to switch to it. Context is preserved between switches.

Delete Session: Remove a session and its entire history using the delete action in the session menu.

4.4 Slash Commands

Slash commands provide quick access to common AI-powered operations directly from the chat input. Type a forward slash (/) to see the available commands:

Command	Description
/run	Execute the current file or a specified command
/edit	Request AI-assisted code editing on the active file
/fix	Automatically diagnose and fix errors in the current file
/refactor	Refactor the selected code for improved readability and maintainability
/optimize	Optimize code performance based on AI analysis
/search	Search across the entire project for patterns, files, or code
/commit	Generate a commit message and stage changes for version control
/explain	Get a detailed explanation of the selected or current code
/status	Display current project status, agent states, and task progress
/deploy	Initiate deployment through Blaze (DevOps agent)
/build	Run the project build pipeline and display results
/test	Execute the test suite and display coverage results
/create_file	Create a new file with AI-generated boilerplate content

Table 5: Slash Commands Reference

5. Code Editor

5.1 Multi-File Tabs

The code editor supports multiple open files displayed as tabs along the top of the editor area. Tabs can be reordered by dragging and dropping, and each tab shows the file name with an unsaved changes indicator (a dot icon) when modifications have not yet been saved. Closing a tab with unsaved changes will prompt a confirmation dialog.

5.2 Syntax Highlighting

TeamForge IDE provides syntax highlighting for a wide range of programming languages and file formats. The editor displays line numbers along the left gutter, and syntax elements such as keywords, strings, comments, and types are color-coded for quick visual parsing. The color scheme follows the Catppuccin Mocha theme for a comfortable, low-eye-strain experience during extended coding sessions.

5.3 Find and Replace

The built-in Find and Replace bar supports the following search modes:

Regular Expression: Toggle regex mode to search using pattern matching.

Case-Sensitive: Toggle case sensitivity for exact-match searching.

Whole-Word: Match only complete words, excluding partial matches.

Replace All: Replace all occurrences at once, or step through matches one at a time.

Activate Find and Replace using **Ctrl+F** for find-only mode, or **Ctrl+H** for find and replace mode.

5.4 Navigation

The editor provides several navigation features to help you move quickly through your codebase:

Go to Line (Ctrl+G): Jump directly to a specific line number in the current file.

Global Search (Ctrl+Shift+F): Search across all files in the project with result preview.

Breadcrumb Navigation: Click on any segment of the breadcrumb path (folder / file / symbol) to navigate.

Tab Switching (Ctrl+Tab): Cycle through open file tabs quickly.

5.5 Run Current File

The Run Current File feature (**Ctrl+Enter**) automatically detects the programming language of the active file and executes it with the appropriate runtime. The output is displayed in the bottom panel's terminal tab. Supported auto-detection includes: JavaScript/TypeScript (Node/Bun), Python, Shell scripts, and more. If the language cannot be determined, the system prompts for manual run command specification.

5.6 Auto-Save

TeamForge IDE features automatic saving of file changes. When auto-save is enabled, modifications are saved after a brief idle period. Files with unsaved changes display a visual indicator (dot icon) on their tab. The auto-save interval can be configured in the Settings panel under Editor Preferences.

6. Project Management

6.1 Virtual File System

The Virtual File System (VFS) provides a complete file management interface within the browser, eliminating the need for local file system access. The VFS supports:

- Create Files:** Add new files with automatic type detection and boilerplate templates.
- Create Directories:** Organize code with nested directory structures.
- Edit Files:** Modify files directly in the code editor; changes are persisted to the VFS.
- Delete Files/Directories:** Remove files or entire directories with confirmation prompts.
- Batch Operations:** Perform create, update, or delete operations on multiple files simultaneously via the batch API endpoint.
- Mkdir:** Create nested directory structures with a single API call.

6.2 Task Board

The integrated task board provides a Kanban-style interface for managing work items across six status columns, mirroring the software development lifecycle:

Status	Description	Agent Assignment
Backlog	Newly created tasks awaiting triage and prioritization	Unassigned
Todo	Prioritized tasks ready to be picked up by an agent	Auto-assigned
In Progress	Tasks currently being worked on by an assigned agent	Assigned agent
In Review	Completed tasks undergoing code review by Prism	Prism (Reviewer)
Done	Tasks that have passed review and are complete	N/A
Blocked	Tasks that cannot proceed due to dependencies or issues	Requires escalation

Table 6: Task Board Status Columns

6.3 Task Priority Levels

Priority	Color	Description
Critical	Red	Must be resolved immediately. Blocks other work or affects production.
High	Orange	Important and should be addressed soon. Significant impact on progress.

Medium	Yellow	Standard priority. No immediate urgency but should be planned.
Low	Green	Nice-to-have items. Can be deferred without impacting core functionality.

Table 7: Task Priority Levels

6.4 Task Types

Type	Description	Typical Agent
Feature	New functionality or enhancement to existing features	Codey (Developer)
Bugfix	Correction of defects or unexpected behavior	Codey (Developer)
Refactor	Code restructuring without changing external behavior	Codey / Prism
Test	Creation or improvement of test coverage	Flux (Tester)
Docs	Documentation writing or updates	Nova (PM)
Infra	Infrastructure or DevOps-related changes	Blaze (DevOps)

Table 8: Task Types and Default Agent Assignments

6.5 Auto-Assignment

The auto-assignment system matches tasks to agents based on task type and agent specialization. When a task is moved to "Todo" status, the orchestrator automatically assigns it to the most appropriate available agent. The assignment logic considers: task type to agent role mapping (as shown in Table 8), current agent workload and status, agent expertise level and past performance on similar tasks, and task priority -- critical and high-priority tasks are assigned first.

7. Terminal and Build

7.1 Interactive Terminal

The built-in interactive terminal allows you to execute commands directly within the IDE. It provides a full command-line interface with the following capabilities: command execution with real-time output streaming, command history navigation using up/down arrow keys, working directory management, and process signal handling (interrupt, kill). The terminal is accessible from the bottom panel and can be toggled with keyboard shortcuts.

7.2 Build, Lint, and Test Commands

TeamForge IDE integrates common development commands directly into the workflow:

Build: Compile the project and report any errors. Results are displayed in the Build Logs tab.

Lint: Run the configured linter (ESLint, Pylint, etc.) and display warnings and errors.

Test: Execute the test suite and show results including pass/fail counts and coverage metrics.

7.3 Pre-Deploy Checks

Before deployment, TeamForge IDE runs a series of pre-deploy checks to ensure code quality and readiness. These checks include: all tests passing with adequate coverage, no critical or high-severity linting errors, successful build completion, no uncommitted changes in the working directory (optional), and dependency security audit (if configured). Pre-deploy checks can be triggered manually or are automatically run by Blaze when initiating deployment.

7.4 Command Whitelist

For security, the terminal operates with a command whitelist that restricts which commands can be executed. The default whitelist includes safe, read-only, and development commands. Commands outside the whitelist require explicit approval. The whitelist can be customized in the Settings panel under Security configuration. This prevents accidental execution of destructive commands such as `rm -rf`, `DROP TABLE`, or other system-altering operations.

8. Settings and Configuration

8.1 AI Provider Settings

Configure your preferred AI provider and model from the Settings panel. Each provider requires specific configuration:

Z-AI: Requires an API key for the Z-AI platform. Select between GLM-4 (high quality) and GLM-4 Flash (low latency).

NVIDIA NIM: Requires an NVIDIA API key. Choose from 80+ models across multiple model families.

OpenAI-Compatible: Requires a base URL and optional API key. Configure the endpoint URL for your self-hosted or third-party service.

8.2 Editor Preferences

Setting	Options	Default
Font Size	10 - 24 pt	14
Tab Size	2, 4, 8 spaces	2

Word Wrap	On / Off	On
Minimap	Enabled / Disabled	Enabled
Auto-Save	Enabled / Disabled	Enabled
Auto-Save Delay	500 - 5000 ms	1000 ms
Line Numbers	On / Off	On
Bracket Pair Colorization	On / Off	On

Table 9: Editor Preference Settings

8.3 API Key Configuration

API keys are stored securely in the application configuration and are never exposed in client-side code. To configure an API key: open Settings, navigate to the AI Provider section, select your provider, enter the API key in the designated field, and click "Test Connection" to verify. API keys are transmitted over HTTPS and stored in encrypted environment variables on the server side.

8.4 Connection Testing

Each AI provider configuration includes a "Test Connection" button that sends a test request to verify: API key validity and authentication, endpoint accessibility and response time, model availability and response quality. A successful test displays a green checkmark along with the response latency. A failed test shows the error message and suggested remediation steps.

9. Keyboard Shortcuts

TeamForge IDE provides a comprehensive set of keyboard shortcuts for efficient workflow. The shortcuts overlay can be toggled at any time by pressing **Ctrl+Shift+?** (or **Cmd+Shift+?** on macOS).

Shortcut	Action
Ctrl+S	Save current file
Ctrl+Z	Undo
Ctrl+Shift+Z	Redo
Ctrl+F	Find in current file
Ctrl+H	Find and Replace
Ctrl+G	Go to Line

Ctrl+Shift+F	Global Search across project
Ctrl+Enter	Run current file
Ctrl+P	Quick file open
Ctrl+Tab	Switch to next tab
Ctrl+Shift+Tab	Switch to previous tab
Ctrl+W	Close current tab
Ctrl+B	Toggle sidebar visibility
Ctrl+J	Toggle bottom panel
Ctrl+Shift+E	Focus file explorer
Ctrl+Shift+P	Open command palette
Ctrl+Shift+?	Show keyboard shortcuts overlay
Ctrl+`	Toggle terminal panel
Ctrl+N	New file
Ctrl+Shift+N	New directory
Ctrl+,	Open Settings
Ctrl+K Ctrl+C	Toggle line comment
Ctrl+K Ctrl+U	Remove line comment
Alt+Up	Move line up
Alt+Down	Move line down

Table 10: Keyboard Shortcuts

10. API Reference

TeamForge IDE exposes a RESTful API that allows programmatic access to all core features. The API is organized into the following resource groups:

Endpoint	Methods	Description
/api/agents	GET, POST	List all agents or create a new agent task
/api/agents/[id]	GET, PATCH, DELETE	Get, update, or delete a specific agent

/api/tasks	GET, POST	List all tasks or create a new task
/api/tasks/[id]	GET, PATCH, DELETE	Get, update, or delete a specific task
/api/chat-sessions	GET, POST	List chat sessions or create a new session
/api/chat-sessions/[id]	GET, PATCH, DELETE	Get, update, or delete a session
/api/ai/chat	POST	Send a message to the AI and receive a streaming response
/api/files	GET, POST	List files or create a new file in the VFS
/api/files/[id]	GET, PATCH, DELETE	Get, update, or delete a specific file
/api/execute	POST	Execute a command in the terminal
/api/projects	GET, POST	List projects or create a new project
/api/projects/[id]	GET, PATCH, DELETE	Get, update, or delete a project
/api/projects/[id]/export	GET	Export project data as a downloadable archive
/api/projects/import	POST	Import project data from an uploaded archive
/api/vfs	GET, POST	Virtual file system read and write operations
/api/vfs/mkdir	POST	Create a directory in the virtual file system
/api/vfs/delete	POST	Delete a file or directory from the VFS
/api/vfs/batch	POST	Batch file operations (create, update, delete multiple)
/api/terminal	POST	Execute terminal commands with output streaming
/api/agent-scheduler	GET, POST	View and manage the agent task scheduler
/api/build-logs	GET	Retrieve recent build log entries
/api/metrics	GET	System performance metrics and agent statistics
/api/dashboard	GET	Dashboard summary data for the overview panel
/api/git	GET, POST	Git operations (status, commit, push, pull)

Table 11: REST API Endpoints

11. AI Providers Reference

11.1 Z-AI

Z-AI is the default AI provider for TeamForge IDE, offering models specifically optimized for software development tasks:

Model	Context Window	Best For	Latency
GLM-4	128K tokens	Complex reasoning, architecture design, multi-step code generation	Standard
GLM-4 Flash	128K tokens	Quick completions, simple edits, rapid prototyping	Low

Table 12: Z-AI Models

11.2 NVIDIA NIM

NVIDIA NIM provides access to over 80 production-grade AI models through a unified API. The following model families are available:

Model Family	Examples	Strengths
Meta Llama	Llama 3.1 8B/70B/405B	General-purpose, strong reasoning, open weights
NVIDIA Nemotron	Nemotron 4 340B	NVIDIA-optimized, high accuracy on code tasks
DeepSeek	DeepSeek-V2, DeepSeek-Coder	Exceptional code generation and understanding
Mistral	Mistral 7B, Mixtral 8x22B	Efficient inference, strong multilingual support
Qwen	Qwen 2 72B, Qwen 1.5 110B	Strong on Asian languages and code
Google Gemma	Gemma 2 9B/27B	Lightweight, efficient for edge deployments
Microsoft Phi	Phi-3 Mini/Small/Medium	Small but capable, ideal for constrained environments

Table 13: NVIDIA NIM Model Families

11.3 OpenAI-Compatible Endpoints

The OpenAI-Compatible provider allows TeamForge IDE to connect to any service that implements the OpenAI API format. Supported platforms include:

Ollama: Run open-source models locally with GPU acceleration. Ideal for air-gapped environments or cost-sensitive workflows. Supports Llama, Mistral, Phi, and many other models.

LM Studio: A desktop application for running local LLMs with a user-friendly interface. Provides OpenAI-compatible API endpoint for easy integration.

vLLM: High-throughput inference server for production deployments. Supports continuous batching and PagedAttention for optimal GPU utilization.

Together AI: Cloud-based inference platform offering access to open-source models at scale. Provides serverless endpoints with auto-scaling.

To configure an OpenAI-Compatible provider, specify the base URL of the API endpoint and optionally provide an API key. The system automatically detects available models from the /v1/models endpoint.

12. Troubleshooting

12.1 Common Issues and Solutions

Issue	Cause	Solution
Application fails to start	Missing dependencies or incorrect Node version	Run bun install and verify Node.js 18+ is installed
Database connection error	PostgreSQL not running or wrong credentials	Check DATABASE_URL in .env and ensure PostgreSQL is running
Blank page on load	JavaScript build error	Clear browser cache and check console for errors
Agents stuck in "Thinking"	AI provider unreachable or rate limited	Check API key validity and provider status; try a different model
File changes not persisting	Auto-save disabled or VFS sync error	Enable auto-save in Settings; refresh the page to re-sync VFS
Terminal commands not executing	Command blocked by whitelist	Check the command whitelist in Security settings and add the command
Slow AI responses	Network latency or large context window	Switch to a faster model (GLM-4 Flash) or reduce context size

Table 14: Common Issues and Solutions

12.2 WebSocket Connection Issues

TeamForge IDE uses WebSocket connections for real-time features such as agent status updates, terminal output streaming, and live chat responses. If you experience connectivity issues:

Check proxy configuration: Ensure your reverse proxy (Nginx, Caddy, etc.) is configured to support WebSocket upgrades with the appropriate headers (Upgrade, Connection).

Verify port access: The WebSocket service runs on a dedicated port. Ensure it is not blocked by firewalls or network policies.

Connection timeout: If the connection drops after idle periods, check the WebSocket keep-alive settings on both client and server.

Multiple tabs: Opening the application in multiple browser tabs creates separate WebSocket connections. Each tab maintains its own session state.

12.3 AI Provider Fallback

When the primary AI provider is unavailable, TeamForge IDE supports automatic fallback to a secondary provider. The fallback chain is configured in Settings under AI Providers. For example, if Z-AI is unreachable, the system can automatically route requests to NVIDIA NIM or your OpenAI-Compatible endpoint. The fallback behavior is as follows: the system attempts the primary provider first with a configurable timeout (default: 30 seconds). On timeout or error, the next provider in the chain is tried. If all providers fail, the user is notified with the specific error from each provider. Fallback events are logged in the System Log panel for debugging purposes.